

MyBatis 페이징 처리

▶ JSP/Servlet 페이징 처리

✓ 게시판 페이지 계산식(Controller)

```
int listCount = 123;    // 게시글 총 갯수          --> 200이라 가정
int currentPage = 1;    // 현재 페이지            --> 1이라 가정
int pageLimit = 10;     // 한 화면에 보여질 페이징 수 --> 10이라 가정
int maxPage;            // 전체 페이징의 마지막 페이지 (총 페이지수)
int startPage;          // 현재 페이지에서 보여질 페이징의 시작 페이지
int endPage;            // 현재 페이지에서 보여질 페이징의 끝 페이지

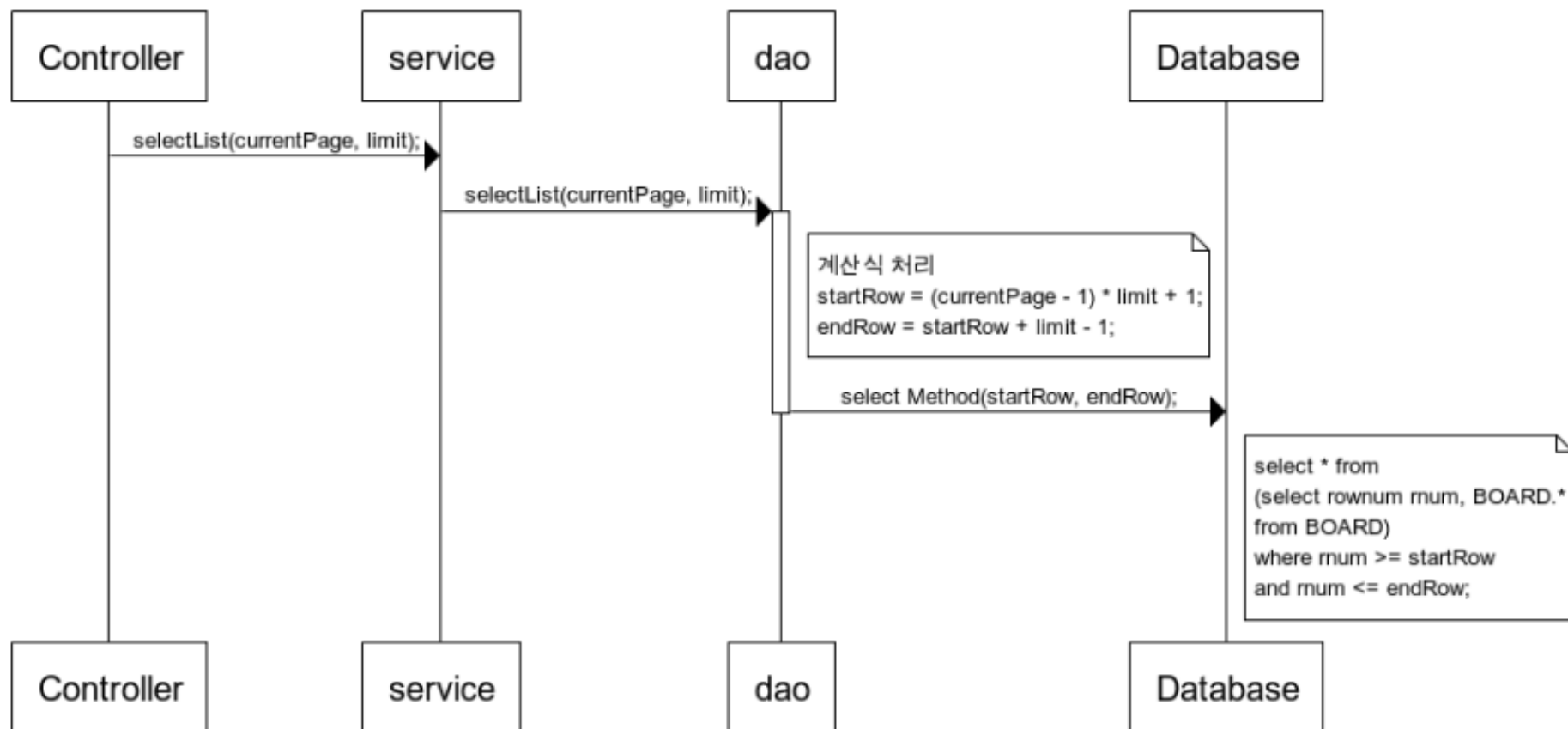
int boardLimit = 5;     // 현재 페이지에서 보여질 게시글 최대 개수

maxPage = (int)(Math.ceil(((double)listCount / boardLimit)));
startPage = ((currentPage - 1) / pageLimit) * pageLimit + 1;
endPage = startPage + pageLimit - 1;

if(endPage > maxPage)
    endPage = maxPage;
```

▶ JSP/Servlet 페이지 처리

✓ 시퀀스 다이어그램



▶ JSP/Servlet 페이징 처리

✓ 페이징 처리(DAO)

- SQL Query 구문 생성

Select *

FROM (SELECT ROWNUM RNUM, BOARD.*,

FROM BOARD

ORDER BY BOARD_DATE DESC)

WHERE RNUM >= ? AND RNUM <= ?

- 구문 실행

```
pstmt = conn.prepareStatement(sql);
```

```
pstmt.setInt(1, startRow);    int startRow = (currentPage - 1) * boardLimit + 1;
```

```
pstmt.setInt(2, endRow);     int endRow = startRow + boardLimit - 1;
```

```
rs = pstmt.executeQuery(pstmt);
```

▶ MyBatis 페이징 처리

✓ RowBounds

MyBatis에서 제공하는 게시판 페이징 처리를 위한 객체

```
int currentPage = 1; // 현재 페이지 1이라 가정
int boardLimit = 5;  // 한페이지에 보여질 게시판 최대 갯수 5라 가정

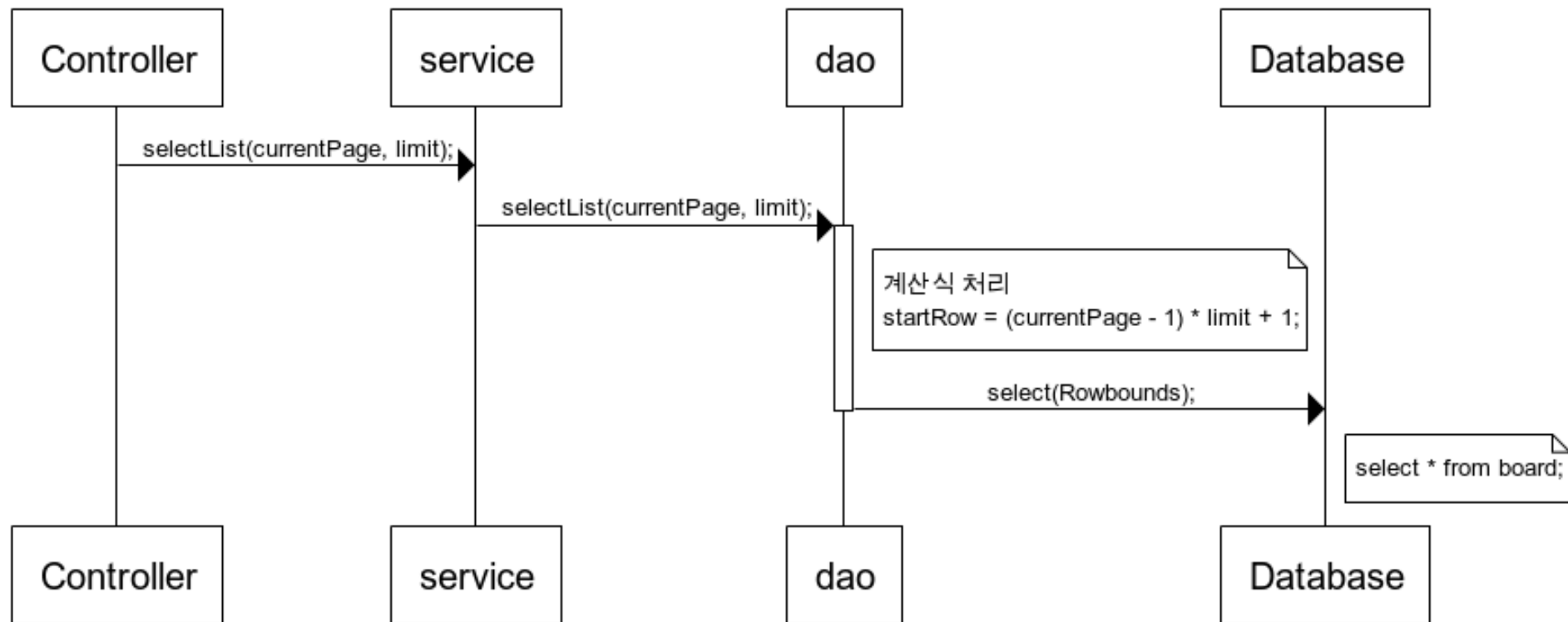
int offset = (currentPage - 1) * boardLimit;

RowBounds rowBounds = new RowBounds(offset, boardLimit);

session.selectList("boardMapper.selectAll", null, rowBounds);
// 넘겨줄 parameter 객체가 없으므로 null을 기입하여 session을 작성한다.
```

▶ MyBatis 페이징 처리

✓ 시퀀스 다이어그램



▶ MyBatis 페이징 처리

✓ 페이징 처리(DAO)

- Mapper에 SQL Query 구문 생성

```
<select>
```

```
    SELECT * FROM BOARD
```

```
</select>
```

- RowBounds 객체 생성

```
int offset = (currentPage - 1) * boardLimit;
```

```
RowBounds rowBounds = new RowBounds(offset, boardLimit);
```

- 구문 실행

```
ArrayList<Board> list = new
```

```
ArrayList<>(session.selectList("boardMapper.selectAll", null, rowBounds));
```

▶ MyBatis 페이징 처리

✓ ROWNUM VS RowBounds

	ROWNUM/ROW_NUMBER	RowBounds
장점	대량의 데이터도 빠르게 페이징 처리하여 가져올 수 있음	구현이 쉽고 간편한 코드 유지보수
단점	페이징 처리 구현 코드 복잡	대량의 데이터를 사용할 경우 수행 속도가 느림